**PROJETO SPRING BOOT**

**MODELAGEM DO BANCO DE DADOS**

```
-- Tabela Professor
CREATE TABLE Professor (
    id_professor INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    departamento VARCHAR(50)
);

-- Tabela Curso
CREATE TABLE Curso (
    id_curso INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    id_professor INT,
    FOREIGN KEY (id_professor) REFERENCES Professor(id_professor)
);

-- Tabela Aluno
CREATE TABLE Aluno (
    id_aluno INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    id_curso INT,
    FOREIGN KEY (id_curso) REFERENCES Curso(id_curso)
);
```

**REST API (EXEMPLO):**

Em uma REST API com Spring Boot, quando uma tabela possui uma chave estrangeira, normalmente o Request DTO não precisa receber o objeto inteiro da tabela relacionada. O mais comum é receber apenas o ID da entidade relacionada.

Por exemplo, imagine estas tabelas:

CLIENTE

---
id
nome

PEDIDO
---
id
data
cliente_id → CLIENTE.id
1. Entidade Cliente
@Entity
@Table(name = "cliente")
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;

    // getters e setters
}
2. Entidade Pedido
A chave estrangeira é representada pelo relacionamento:
@Entity
@Table(name = "pedido")
public class Pedido {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private LocalDate data;

    @ManyToOne

```
    @JoinColumn(name = "cliente_id")
    private Cliente cliente;

    // getters e setters
}
```

3. Classe PedidoRequest

Aqui está o ponto principal. Eu recomendaria:

```
public class PedidoRequest {

    private LocalDate data;

    private Long clienteId;

    // getters e setters
}
```

Ou seja, o JSON enviado para a API seria:

```
{
    "data": "2026-09-08",
    "clienteId": 10
}
```

Você não precisa enviar:

```
{
    "data": "2026-09-08",
    "cliente": {
        "id": 10,
        "nome": "João"
    }
}
```

4. No Service

O Service recebe o Request e procura o cliente pelo ID:

```
public Pedido criar(PedidoRequest request) {

    Cliente cliente = clienteRepository
            .findById(request.getClienteId())
            .orElseThrow(() -> new RuntimeException("Cliente não encontrado"));
```

```
    Pedido pedido = new Pedido();

    pedido.setData(request.getData());
    pedido.setCliente(cliente);

    return pedidoRepository.save(pedido);
}
```